23CSE455 Design Patterns
Part 2 Findings: Evolution and Refactoring

Design Pattern Identification and Evolution in a Spring MVC-Based Warehouse Management Application

Team: B9
Date: 11 October 2026 (re-audited findings)
Team working repository: akdswordguy/Team_B9_DP
Companion documents: `report/main.tex` (full report), `report/part1_findings.tex`, `report/part3_findings.tex`

Roll number	Student name	Batch
AM.SC.U4CSE23215	D Aditya Kiran	CSE C
AM.SC.U4CSE23235	Mithun Krishna	CSE C
AM.SC.U4CSE23248	Sai Kishen K M	CSE C
AM.SC.U4CSE23359	Sivadev Smijith	CSE D

Evidence paths in this document are relative to `Team_B7_Findings/Part2/` in the team working repository unless stated otherwise.

1 Overview

Change	Where	Nature	Tests
CH01 Multiple-Warehouse Support	Upstream `org.openwms.common.service.lib` and `org.openwms.common.service`	Uncommitted modification of real OpenWMS repositories	187 run, 7 skipped (Library); 1 (Service) — recorded
CH02 Barcode-Based Picking	`CH02-barcode-picking/`	Standalone Spring Boot reference implementation	14/14 — recorded; 14 `@Test` in source
CH03 Automatic Stock Reordering	`CH03-stock-reorder/`	Standalone Spring Boot reference implementation	19/19 — recorded; 19 `@Test` in source

CH02 and CH03 are reference implementations because the canonical modules were not accessible: the OpenWMS CORE README lists the Picking Library as a private GPLv3 repository and the Inventory Service as private preview. None of the three changes is an upstream merged change, and none introduces a new GoF pattern.

Re-audit method (11 October 2026). CH01 was checked by diffing the published complete-source snapshots against fresh clones of the two upstream repositories at their baselines. CH02 and CH03 were checked by recounting files, physical Java LOC and `@Test` methods in the published source, reading every test, and confirming that each acceptance claim matches code. Maven could not be re-run because Maven Central was blocked in the audit environment, so build and test totals marked "recorded" are the team's original runs.

2 CH01 — Multiple-Warehouse Support

2.1 Requirement

Introduce explicit warehouse identity so that locations with identical coordinates can exist in different warehouses without collision. A location belongs to one warehouse; its identity is the warehouse plus the existing five-part `LocationPK` (area, aisle, x, y, z). Coordinate uniqueness is warehouse-scoped; PLC and ERP codes stay globally unique. `LocationGroup` remains a logical grouping and does not become a warehouse. An unqualified lookup that matches several warehouses must not silently pick one.

2.2 Provenance and baselines

Repository	Baseline commit	CH01 state
openwms/org.openwms.common.service.lib	a6566d7a90364f8cb0e82ec013720b644926dffe	31 modified, 13 new files; uncommitted
openwms/org.openwms.common.service	fc7edd4ffbe16de65d482b3a2d462d4d896b1998	2 modified files; uncommitted

Re-audit result. Diffing `CH01-multiple-warehouses/*.complete-source/` against upstream reproduced the changed-file lists in `PROVENANCE.md` exactly. The Library grew from 226 to 238 Java files (12 new Java files plus the new `9-warehouse.adoc`). The partial evidence snapshots are byte-identical to the complete-source copies, apart from a UTF-8 byte-order mark on one `README.md`. Credential-bearing files were excluded from the copies, so the snapshots are source-complete but not guaranteed standalone builds.

2.3 Design change

- New `org.openwms.common.warehouse` package: `Warehouse` entity, `WarehouseController`, `WarehouseService`/`WarehouseServiceImpl`, `WarehouseRepository`, `WarehouseMapper`, `WarehouseApi`, `WarehouseVO`.
- `Location` gains a mandatory (`nullable=false`) `Warehouse` reference, and the `UC_LOC_ID` unique constraint changes from (`C_AREA`, `C_AISLE`, `C_X`, `C_Y`, `C_Z`) to (`C_WAREHOUSE`, `C_AREA`, `C_AISLE`, `C_X`, `C_Y`, `C_Z`). `UC_LOC_PLC_CODE` and `UC_LOC_ERP_CODE` are unchanged.
- Location and transport-unit services, controllers, APIs, the `LocationMO` message and the command handler gain warehouse-qualified operations; unqualified operations still work when the coordinate is unique and are rejected when it is ambiguous.
- No default warehouse and no data migration are added.

2.4 Acceptance criteria and test evidence

The CH01 tests are identifiable by name: 18 test methods were added (245 `@Test` annotations at the baseline, 263 after).

Criterion	Test methods (class.method)	Status
Same coordinates in different warehouses	`LocationTest.same_coordinates_in_different_warehouses_are_distinct_locations`; `LocationIT.shall_allow_same_coordinates_in_different_warehouses`; `LocationServiceImplIT.shall_allow_same_location_id_in_different_warehouses_and_find_by_warehouse`	Recorded as passed
Duplicate coordinates in one warehouse	None. Enforced by the `UC_LOC_ID` constraint only.	Enforced by schema; not tested
Warehouse-qualified lookup and update	`TransportUnitServiceImplIT.update_with_warehouse_code_selects_the_requested_warehouse_for_duplicate_coordinates`; `TransportUnitCommandHandlerIT.creates_transport_units_at_the_same_coordinates_in_the_requested_warehouses`	Recorded as passed
Ambiguous unqualified lookup rejected	`TransportUnitServiceImplIT.update_without_warehouse_code_rejects_an_ambiguous_coordinate`; `TransportUnitCommandHandlerIT.rejects_unqualified_creation_when_the_coordinate_is_ambiguous`	Recorded as passed
Unique unqualified lookup still works	`TransportUnitServiceImplIT.update_without_warehouse_code_resolves_a_unique_coordinate`; `TransportUnitCommandHandlerIT.creates_without_warehouse_when_the_coordinate_is_unique`	Recorded as passed
Unknown warehouse rejected	`LocationServiceImplIT.shall_reject_location_with_unknown_warehouse`; `WarehouseServiceImplIT.rejects_an_unknown_warehouse_code`	Recorded as passed
Warehouse management	`WarehouseServiceImplIT.creates_and_retrieves_a_warehouse`, `rejects_a_duplicate_warehouse_code`	Recorded as passed
API/message carry warehouse code	`LocationServiceImplIT.location_api_and_message_carry_warehouse_code`; `LocationMOTest.warehouse_code_is_preserved_by_json_serialization`	Recorded as passed
Legacy `LocationMO` constructor	`LocationMOTest.legacy_nine_argument_constructor_preserves_null_warehouse`	Recorded as passed
PLC/ERP global uniqueness	None. `UC_LOC_PLC_CODE`/`UC_LOC_ERP_CODE` unchanged from baseline.	Preserved by schema; not tested
`LocationGroup` semantics	Source/design review.	Preserved by design
Production data rollout	No migration/backfill supplied.	Not completed

2.5 Recorded verification

- Library: Maven 3.9.9 `mvn.cmd -DforkCount=0 install` — `BUILD SUCCESS`; 187 tests, 0 failures, 0 errors, 7 skipped. Asciidoctor logged errors for an unavailable generated snippet and invalid part structure despite the successful status.
- Common Service: `mvn.cmd -DforkCount=0 verify` — `BUILD SUCCESS`; 1 test, 0 failures/errors/skips; Failsafe reported no integration tests.
- `git diff --check` exited 0 in both CH01 repositories and in CORE.

2.6 Pattern impact, metrics and limitations

No new GoF pattern is introduced; the Part 1 Observer and Composite instances are untouched, and the generic `TreeNode` Composite does not become a warehouse hierarchy. No before/after LOC metrics were recorded and none are estimated. The changes remain uncommitted, so there is no after commit. Because `C_WAREHOUSE` is non-nullable, existing production locations need an explicit warehouse assignment before rollout; no Flyway/Liquibase migration or backfill is supplied, and ORM auto-update is not a substitute.

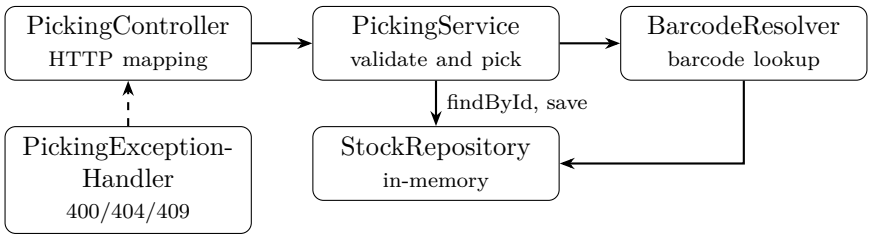
3 CH02 — Barcode-Based Picking

3.1 Requirement and design

Resolve a scanned barcode to stock, validate the item and quantity, pick, and return the updated stock.

Before (`5ca90cc`): `PickingController` → `PickingService.pickByItemId` → `StockRepository.findById` → `StockItem.pick`, exposed as `POST/api/picking/item`.

After (`0f0fbf0`): `PickingController.pickByBarcode` → `PickingService.pickByBarcode` → `BarcodeResolver.resolve` → `StockRepository.findByBarcode`; the service validates quantity, calls `StockItem.pick`, saves, and returns a `PickingResult`. `PickingExceptionHandler` maps `InvalidPickingQuantityException` to 400, `BarcodeNotFoundException` to 404, and `InsufficientStockException`/`StockUnavailableException` to 409. `PickingRequest` validates `@NotBlankbarcode` and `@Min(1)quantity`. The direct item-ID route is kept.



3.2 Acceptance criteria and test evidence

ID	Criterion	Test methods	Result
AC1	Valid barcode resolves and pick succeeds	`PickingControllerTest.post_valid_barcode_picks_stock_and_returns_result`; `PickingServiceTest.picks_stock_resolved_from_a_barcode_and_decreases_quantity`; `BarcodeResolverTest.resolves_an_existing_barcode_to_its_stock_item`	Pass
AC2	Unknown barcode rejected without creating stock	`BarcodeResolverTest.rejects_an_unknown_barcode_without_creating_stock`; `PickingControllerTest.post_unknown_barcode_returns_not_found`	Pass
AC3	Unavailable stock cannot be picked	`PickingServiceTest.rejects_unavailable_stock`; `PickingControllerTest.post_unavailable_stock_returns_conflict`	Pass
AC4	Quantity above stock rejected	`PickingServiceTest.rejects_quantity_greater_than_available_stock`; `PickingControllerTest.post_excessive_quantity_returns_conflict`	Pass
AC5	Successful pick decreases and returns remaining quantity	`PickingServiceTest.picks_stock_resolved_from_a_barcode_and_decreases_quantity`	Pass
AC6	Zero and negative quantities rejected	`PickingServiceTest.rejects_zero_and_negative_quantities`; `PickingControllerTest.post_zero_quantity_returns_bad_request`	Pass
AC7	Direct item-ID picking still works separately	`PickingServiceTest.picks_a_requested_quantity_by_stock_item_id`, `direct_item_id_picking_remains_available_separately`; `PickingControllerTest.direct_item_id_endpoint_remains_separate_and_available`	Pass
AC8	Endpoint returns documented statuses	All six `PickingControllerTest` methods (200, 400, 404, 409)	Pass

Recorded: `mvn.cmd -DforkCount=0 verify` — `BUILD SUCCESS`, 14 tests, 0 failures. The source contains exactly 14 `@Test` methods (6 controller, 2 resolver, 6 service). Ignored, generated `PickingControllerIT` Surefire files from a deleted test source are stale and excluded.

3.3 Metrics

State	Prod. files	Test files	Total files	Java LOC	Tests
Before (`5ca90cc`)	7	1	8	197	1
After (`0f0fbf0`)	14	3	17	526	14

LOC is physical lines including blanks and comments. The after row was re-measured from the published source and matches. The before row cannot be re-measured from the team repository (Section 5).

3.4 Pattern impact and limitations

No new GoF pattern. MVC separation and the repository interface are architectural mechanisms, and `BarcodeResolver` is a focused collaborator, not Strategy or Adapter. Storage is an in-memory demo with one seeded item (barcode `8901234567890`); there is no database, scanner hardware or external integration.

4 CH03 — Automatic Stock Reordering

4.1 Requirement and design

Add per-item reorder rules and automatically create a reorder request when consumption brings stock to or below the threshold. A reorder is created when `availableQuantity<=reorderThreshold` and `availableQuantity<targetQuantity`; the requested quantity is `targetQuantity-availableQuantity`. Rules reject a blank item ID, a negative threshold, and a target not greater than the threshold.

Before (`e9ad4c1`): `StockController` → `StockService.consume` → `StockRepository` → `StockItem.consume`; no reorders.

After (`59b1b59`): after a successful consume, `StockService` calls `ReorderService.evaluate(itemId)`, which loads stock and rule, returns an existing OPEN request if present, and otherwise creates one when the rule fires. `evaluate` and `complete` are `synchronized`. `ReorderController` exposes evaluate (200, or 204 when no reorder is needed), list, get and complete; `ReorderRuleController` manages rules. Completing a request adds its quantity back to stock and marks it COMPLETED; completing it again is rejected (409).

4.2 Acceptance criteria and test evidence

ID	Criterion	Test methods	Result
AC1	Stock above threshold: no reorder	`ReorderRuleTest.stock_above_threshold_does_not_require_reorder`; `ReorderServiceTest.stock_above_threshold_does_not_create_a_request`	Pass
AC2	Stock equal to threshold: reorder	`ReorderRuleTest.stock_equal_to_threshold_requires_reorder_to_target`; `ReorderServiceTest.stock_equal_to_threshold_creates_a_request_for_the_gap_to_target`	Pass
AC3	Stock below threshold: reorder	`ReorderRuleTest.stock_below_threshold_calculates_quantity_to_target`; `ReorderServiceTest.stock_below_threshold_creates_a_request_for_the_gap_to_target`	Pass
AC4	Quantity = target − available	Same tests as AC2 and AC3 (they assert the gap)	Pass
AC5	OPEN request reused	`ReorderServiceTest.repeated_evaluation_reuses_the_open_request`	Pass
AC6	Invalid rules rejected	`ReorderRuleTest.rejects_negative_threshold`, `rejects_target_that_is_not_greater_than_threshold`; `StockReorderControllerTest.invalid_rule_is_rejected`	Pass
AC7	Completion replenishes stock	`ReorderServiceTest.automatic_reorder_after_consumption_and_completion_allow_a_future_request`; `StockReorderControllerTest.completing_request_replenishes_stock_and_allows_a_later_reorder`	Pass
AC8	Later low stock creates a new request	Same two tests as AC7	Pass
AC9	Consumption triggers evaluation	`StockReorderControllerTest.consuming_stock_automatically_creates_a_reorder_that_can_be_listed`; `ReorderServiceTest.automatic_reorder_after_consumption_and_completion_allow_a_future_request`	Pass
AC10	Unknown stock rejected	`ReorderServiceTest.unknown_stock_item_is_rejected_when_evaluating`; `StockServiceTest.unknown_item_is_rejected`; `StockReorderControllerTest.unknown_stock_item_is_not_found`	Pass
AC11	Requests retrievable and listable	`ReorderServiceTest.reorder_request_can_be_retrieved_by_id_and_as_a_list`	Pass
—	Repeated completion rejected	`ReorderServiceTest.completing_an_already_completed_request_is_rejected`	Pass

Recorded: `mvn.cmd -DforkCount=0 verify` — `BUILD SUCCESS`, 19 tests, 0 failures. The source contains exactly 19 `@Test` methods (`ReorderRuleTest` 5, `ReorderServiceTest` 8, `StockReorderControllerTest` 4, `StockServiceTest` 2).

Test-quality observation. `StockServiceTest.consumption_reduces_stock_without_creating_a_reorder` configures no rule and asserts only the remaining quantity; it does not check that

no request was created. The no-reorder behavior is still covered by the AC1 tests.

4.3 Metrics

State	Prod. files	Test files	Total files	Java LOC	Tests
Before (`e9ad4c1`)	6	1	7	169	2
After (`59b1b59`)	24	4	28	978	19

The after row was re-measured from the published source and matches.

4.4 Pattern impact and limitations

No new GoF pattern. `ReorderRule` encapsulates one fixed predicate and calculation; with no family of interchangeable algorithms it is not Strategy. Stock, rules, requests and IDs are in memory and lost on restart, and `synchronized` only serializes calls inside one process. There is no supplier, purchase-order, ERP or messaging integration.

5 Commit availability

The CH02 and CH03 before/after commit IDs (`5ca90cc/0f0fbf0` and `e9ad4c1/59b1b59`) refer to the original local project repositories. Those histories were not copied into the team repository, which contains only the after source trees. As a result, the commits cannot be checked out from the submission, and the before metrics cannot be reproduced from it. Publishing the two project histories (for example as Git bundles or separate repositories) would make them verifiable.

6 Pattern counts after the evolutions

Snapshot / change	Types	Instances	Concrete classes	Rejected candidates	Notes
Part 1 baseline	2	2	2	2	Observer and generic Composite confirmed.
After CH01	2	2	2	2	No new GoF pattern; changes uncommitted.
After CH02	2	2	2	2	Separate reference implementation; no new GoF pattern.
After CH03	2	2	2	2	Separate reference implementation; no new GoF pattern.

These counts carry the frozen Part 1 inventory forward for consistency; they are not a fresh pattern scan of each reference project.

7 Corrections made in the re-audit

Item	Previously	Now
CH01 test evidence	"Exact test names/count not retained."	18 named test methods mapped to criteria.
CH01 duplicate coordinates in one warehouse	"Recorded as passed."	Enforced by `UC_LOC_ID`; no test exercises it.
CH01 PLC/ERP uniqueness	"Evidence recorded; test mapping incomplete."	Constraints unchanged from baseline; not tested.
CH03 test evidence	"Source evidence does not supply individual test method names."	All 19 methods named and mapped to AC1–AC11.
CH02/CH03 commits	Presented as checkable evidence.	Noted as local history not included in the team repository.
CH03 typo	"textended"	"extended"

8 Evidence files

- `CH01-multiple-warehouses/PROVENANCE.md` — baselines, changed-file lists, exclusions, recorded verification, re-audit test mapping.
- `CH02-barcode-picking/CH02_EVIDENCE.md`, `CH03-stock-reorder/CH03_EVIDENCE.md` — design, API, criteria, tests, metrics, limitations, re-audit notes.
- `CH03-stock-reorder/CH03-class-diagram.puml` — CH03 class relationships.
- `Team_B7_Findings/LLM_RECORDS/LLM_RECORDS.md` — records CH01-01, CH01-02, the CH01 verification addendum, CH02-01, CH03-01 and AUDIT-01.